

CSE-3203 Final Examination 2022

Design and Analysis of Algorithms-II

Complete Question & Answer Notes

University of Dhaka, Department of Computer Science and Engineering

Exam Tip

The questions are transcribed from the two supplied exam-page screenshots. The answers are worked study solutions using standard algorithms from the course. Where a scan is slightly unclear, the interpretation used is stated explicitly.

Contents

1	Question 1 - Hashing	2
2	Question 2 - String Matching and Tries	3
3	Question 3 - Computational Geometry	5
4	Question 4 - Number Theory and RSA	6
5	Question 5 - Backtracking and Branch and Bound	7
6	Question 6 - NP Problems, Reductions, and Circle Geometry	10
7	Question 7 - Online Algorithms and List Update	12
	Last-Minute Revision Sheet	14

1 Question 1 - Hashing

Q1(a) [4 marks]

Describe the differences between **Linear Probing** and **Quadratic Probing** with proper examples.

Answer

Both are open-addressing methods: all keys are stored inside the hash table and a probe sequence is used after a collision.

Property	Linear probing	Quadratic probing
Probe rule	$h(k, i) = (h'(k) + i) \bmod m$	$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$; common simple form $h'(k) + i^2$
Step sizes	1, 1, 1, ...	Increasing/nonlinear offsets
Clustering	Suffers from primary clustering	Avoids primary clustering, though secondary clustering may remain
Coverage	With unit step, all slots are eventually examined	Choice of m, c_1, c_2 matters; a poor choice may not visit every slot
Implementation	Very simple, cache friendly	Slightly more arithmetic

Example. Let $m = 7$, $h'(k) = k \bmod 7$, and insert 10, 17, 24. All three initially hash to slot 3. Linear probing gives

$$10 \rightarrow 3, \quad 17 \rightarrow 4, \quad 24 \rightarrow 5.$$

Quadratic probing with i^2 gives

$$10 \rightarrow 3, \quad 17 \rightarrow (3 + 1^2) = 4,$$

and for 24, slots 3 and 4 are occupied, so

$$24 \rightarrow (3 + 2^2) \bmod 7 = 0.$$

Thus quadratic probing spreads colliding keys more widely.

Q1(b) [2 marks]

The hash table has indices 0 through 8 and currently contains

$$H[1] = 11, \quad H[2] = 101, \quad H[4] = 22, \quad H[5] = 50, \quad H[7] = 34.$$

For a new number n , its initial index is

$$\text{idx} = n \bmod 9.$$

Identify which of the following entries creates a collision (multiple entries for the same index):

$$600, \quad 666, \quad 682, \quad 701.$$

Answer

Compute each initial index:

$$\begin{aligned} 600 \bmod 9 &= 6, & 666 \bmod 9 &= 0, \\ 682 \bmod 9 &= 7, & 701 \bmod 9 &= 8. \end{aligned}$$

The only occupied one among these indices is index 7, which already contains 34. Therefore

$$\boxed{682 \text{ creates a collision at index 7.}}$$

The other three map to empty slots 6, 0, 8.

Q1(c) [8 marks]

The scan gives the string

$$S = \text{aaaaacba fadfcbb}.$$

For every consecutive substring of length 3, compute

$$H_{i,i+2} = (\text{ASCII}(S[i])B^2 + \text{ASCII}(S[i+1])B + \text{ASCII}(S[i+2])) \bmod M,$$

where $B = 7$ and $M = 13$. Identify whether two *different* three-character sequences can have the same hash value.

Answer

Here $B^2 = 49$. Evaluating every length-3 window gives:

i	substring	hash	i	substring	hash
0	aaa	4	7	afa	0
1	aaa	4	8	fad	5
2	aaa	4	9	adf	4
3	aac	6	10	dfc	6
4	acb	6	11	fc b	4
5	cba	5	12	cbb	6
6	baf	6			

The repeated **aaa** windows are the same sequence, so they are not a collision between different strings. But several *different* triples share the same hash:

$h = 4 : \text{aaa, adf, fcb}$

$h = 5 : \text{cba, fad}$

$h = 6 : \text{aac, acb, baf, dfc, cbb}$

Hence this hash function definitely has collisions.

2 Question 2 - String Matching and Tries

Q2(a) [4 marks]

A pattern is called **nonoverlappable** when it has no nonempty proper prefix that is also a suffix. Give an example and describe the state-transition diagram of the string-matching automaton for a nonoverlappable pattern.

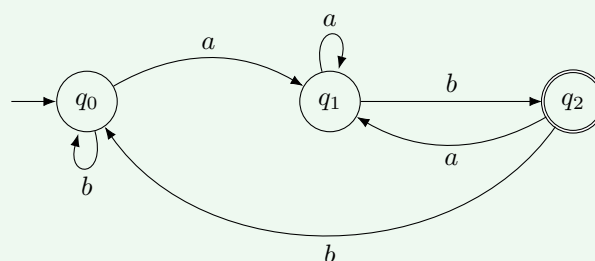
Answer

A simple example is

$$P = \text{ab.}$$

Its only nonempty proper prefix is **a**, while its only length-1 suffix is **b**; hence they are different and the pattern cannot overlap with itself.

For alphabet $\Sigma = \{a, b\}$, the string-matching automaton has states q_0, q_1, q_2 , where state q means that the longest prefix of P matching a suffix of the text read so far has length q . State q_2 is accepting.



Transition table:

	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_1	q_0

The important nonoverlap property is that after a full match there is no positive-length border of the whole pattern to reuse.

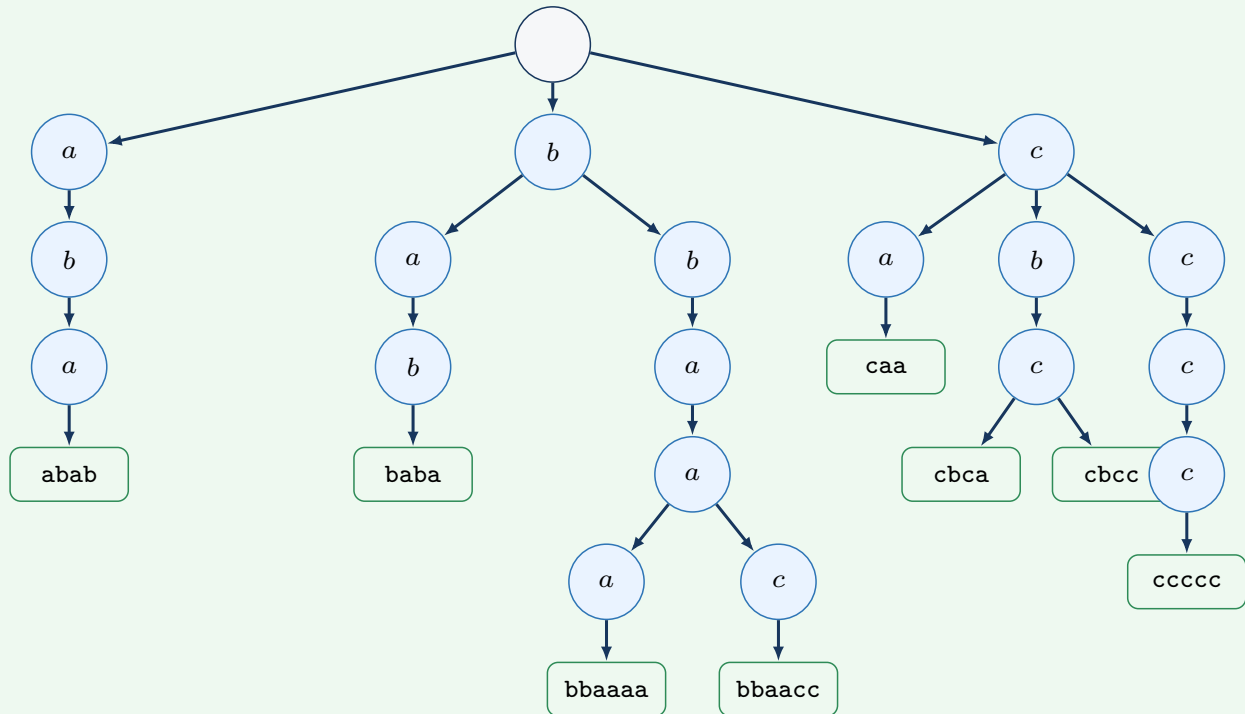
Q2(b) [4 marks]

Draw a standard trie for

$\{\text{abab}, \text{baba}, \text{ccccc}, \text{bbaaaa}, \text{caa}, \text{bbaacc}, \text{cbcc}, \text{cbca}\}.$

Answer

A standard trie uses one character per edge. Green boxes mark complete words.

**Q2(c) [6 marks]**

Given a string X of length n and a string Y of length m , describe an $O(n + m)$ -time algorithm for finding the longest prefix of X that is a suffix of Y .

Answer

Use the KMP prefix function on one combined string.

Choose a separator symbol $\#$ that occurs in neither string, and form

$$Z = X\#Y.$$

Compute the prefix function π for Z in linear time. The value at the last position,

$$\pi[|Z|],$$

is exactly the length of the longest prefix of X that is also a suffix of Y .

Why it works: any prefix-suffix match at the end of Z must start at the beginning of X . The separator prevents a match from crossing from the end of X into the beginning of Y .

```

LongestPrefixSuffix(X, Y):
  Z = X + "#" + Y
  pi = PrefixFunction(Z)
  L = pi[len(Z)-1]
  return X[0 : L]
  
```

Since $|Z| = n + m + 1$ and KMP computes π in linear time,

$$T(n, m) = O(n + m).$$

3 Question 3 - Computational Geometry

Q3(a) [3 marks]

Prove that if $p_1 \times p_2$ is positive, then vector p_1 is clockwise from vector p_2 with respect to the origin $(0,0)$, and if the cross product is negative, then p_1 is counterclockwise from p_2 .

Answer

Write the vectors in polar form:

$$p_1 = r_1(\cos \theta_1, \sin \theta_1), \quad p_2 = r_2(\cos \theta_2, \sin \theta_2).$$

Then

$$\begin{aligned} p_1 \times p_2 &= r_1 r_2 (\cos \theta_1 \sin \theta_2 - \sin \theta_1 \cos \theta_2) \\ &= r_1 r_2 \sin(\theta_2 - \theta_1). \end{aligned}$$

Because $r_1 r_2 > 0$, the sign is determined by $\sin(\theta_2 - \theta_1)$.

If $p_1 \times p_2 > 0$, rotating counterclockwise from p_1 reaches p_2 through the smaller angle; equivalently p_1 is clockwise from p_2 . If the cross product is negative, the relative orientation reverses. If it is zero, the vectors are collinear.

Key Result

For the ordering used in the question:

$$p_1 \times p_2 > 0 \Rightarrow p_1 \text{ clockwise from } p_2$$

$$p_1 \times p_2 < 0 \Rightarrow p_1 \text{ counterclockwise from } p_2.$$

Q3(b) [6 marks]

For the crossing segments shown, with p_1 upper-left, p_2 lower-right, p_3 lower-left and p_4 upper-right, determine whether each expression is positive or negative:

- i. $(p_1 - p_3) \times (p_4 - p_3)$,
- ii. $(p_3 - p_1) \times (p_2 - p_1)$,
- iii. $(p_4 - p_1) \times (p_2 - p_1)$,
- iv. $(p_2 - p_3) \times (p_4 - p_3)$.

Answer

Only the signs are needed, so we may use any coordinates preserving the drawing, for example

$$p_1 = (-1, 1), \quad p_2 = (1, -1), \quad p_3 = (-1, -1), \quad p_4 = (1, 1).$$

Then:

$$\begin{aligned} (p_1 - p_3) \times (p_4 - p_3) &= (0, 2) \times (2, 2) = -4 < 0, \\ (p_3 - p_1) \times (p_2 - p_1) &= (0, -2) \times (2, -2) = 4 > 0, \\ (p_4 - p_1) \times (p_2 - p_1) &= (2, 0) \times (2, -2) = -4 < 0, \\ (p_2 - p_3) \times (p_4 - p_3) &= (2, 0) \times (2, 2) = 4 > 0. \end{aligned}$$

Therefore

$$\text{i} < 0, \quad \text{ii} > 0, \quad \text{iii} < 0, \quad \text{iv} > 0.$$

Q3(c) [5 marks]

Write the pseudocode of Graham Scan to generate a convex hull from a set of points. Show two cases (with diagram): (i) include a point as a hull vertex and (ii) exclude a point from the hull vertex set.

Answer

Graham Scan.

```

GRAHAM-SCAN(P):
  p0 = point with minimum y
    (break tie by minimum x)
  sort remaining points by increasing polar angle about p0
  for equal angles keep only the farthest point

  S = empty stack
  push p0
  push first sorted point
  push second sorted point

  for each remaining point p:
    while size(S) >= 2 and
      orientation(nextToTop(S), top(S), p) <= 0:
      pop(S)
    push(S, p)

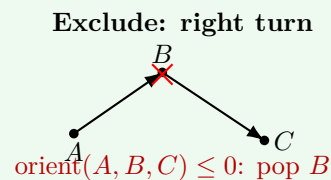
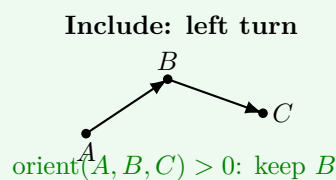
  return S

```

Using the usual orientation

$$\text{orient}(A, B, C) = (B - A) \times (C - A),$$

a positive value means a left turn.



Thus Graham Scan keeps points that preserve the counterclockwise boundary and removes points producing a right turn (or an unwanted collinear interior point).

4 Question 4 - Number Theory and RSA

Q4(a) [4 marks]

Compute the values d, x, y for $(899, 493)$ using the Extended Euclidean Algorithm, where

$$d = \gcd(899, 493) = 899x + 493y.$$

Answer

Euclid's algorithm:

$$899 = 1(493) + 406,$$

$$493 = 1(406) + 87,$$

$$406 = 4(87) + 58,$$

$$87 = 1(58) + 29,$$

$$58 = 2(29) + 0.$$

Hence

$$\boxed{d = 29}.$$

Back-substitute:

$$29 = 87 - 58 = 87 - (406 - 4 \cdot 87) = 5 \cdot 87 - 406,$$

$$= 5(493 - 406) - 406 = 5 \cdot 493 - 6 \cdot 406,$$

$$= 5 \cdot 493 - 6(899 - 493) = 11 \cdot 493 - 6 \cdot 899.$$

Therefore

$$\boxed{d = 29, \quad x = -6, \quad y = 11.}$$

Check:

$$899(-6) + 493(11) = 29.$$

Q4(b) [3 marks]

Find Euler's totient function $\phi(7000)$.

Answer

Factorise:

$$7000 = 2^3 \cdot 5^3 \cdot 7.$$

Then

$$\phi(7000) = 7000 \left(1 - \frac{1}{2}\right) \left(1 - \frac{1}{5}\right) \left(1 - \frac{1}{7}\right).$$

Thus

$$\phi(7000) = 7000 \cdot \frac{1}{2} \cdot \frac{4}{5} \cdot \frac{6}{7} = \boxed{2400}.$$

Q4(c) [7 marks]

Consider an RSA key set with $p = 17$, $q = 11$, $n = 187$, and $e = 7$. Determine the public and private keys. What is the encryption of message $M = 88$? Also prove by decryption that the recovered message is 88.

Answer

First,

$$\phi(n) = (17 - 1)(11 - 1) = 16 \cdot 10 = 160.$$

The private exponent d satisfies

$$7d \equiv 1 \pmod{160}.$$

Since

$$7 \cdot 23 = 161 \equiv 1 \pmod{160},$$

we get

$$\boxed{d = 23}.$$

Therefore:

$$\boxed{\text{Public key } (e, n) = (7, 187)}$$

$$\boxed{\text{Private key } (d, n) = (23, 187)}.$$

Encryption.

$$C = M^e \bmod n = 88^7 \bmod 187.$$

Use repeated squaring:

$$88^2 \equiv 77 \pmod{187}, \quad 88^4 \equiv 132 \pmod{187}.$$

Hence

$$88^7 = 88^4 \cdot 88^2 \cdot 88 \equiv 132 \cdot 77 \cdot 88 \equiv \boxed{11} \pmod{187}.$$

So the ciphertext is $C = 11$.

Decryption.

$$M' = C^d \bmod n = 11^{23} \bmod 187.$$

Now

$$11^2 \equiv 121, \quad 11^4 \equiv 55, \quad 11^8 \equiv 33, \quad 11^{16} \equiv 154 \pmod{187}.$$

Since $23 = 16 + 4 + 2 + 1$,

$$11^{23} \equiv 154 \cdot 55 \cdot 121 \cdot 11 \equiv \boxed{88} \pmod{187}.$$

Thus decryption recovers the original message.

5 Question 5 - Backtracking and Branch and Bound**Q5(a) [3 marks]**

Distinguish between backtracking and branch and bound.

Answer

Aspect	Backtracking	Branch and Bound
Main goal	Find feasible/all solutions to constraint problems	Find an optimal solution to an optimisation problem
Pruning	Reject a partial solution that violates constraints or cannot become feasible	Reject a node whose bound cannot beat the current best solution
Typical search	Depth-first recursion	Best-first, BFS, or other live-node strategy
Main quantity	Promising/feasibility test	Lower or upper bound plus incumbent
Examples	N-Queens, subset sum, graph colouring, Hamiltonian cycle	0/1 knapsack, TSP, assignment
Stopping	May stop after one feasible solution if only one is needed	Must establish that no live node can improve the incumbent

Memory rule:

Backtracking prunes infeasible branches; B&B also prunes noncompetitive branches.

Q5(b) [5 marks]

Given the set

$$S = \{5, 10, 12, 13, 15, 18\}$$

and target sum $m = 30$, draw the state-space tree to print subsets whose sum equals 30 using backtracking. Also define the bounding functions.

Answer

Let the numbers be sorted increasingly. Use a binary decision at each level: include or exclude the next item. A node stores

$$(s, r),$$

where s is the current selected sum and r is the sum of still-available elements. Initially

$$(s, r) = (0, 73).$$

The three solution subsets are

$$\boxed{\{5, 10, 15\}}, \quad \boxed{\{5, 12, 13\}}, \quad \boxed{\{12, 18\}}.$$

For example, one important branch is

$$0 \xrightarrow{+5} 5 \xrightarrow{+10} 15 \xrightarrow{-12} 15 \xrightarrow{-13} 15 \xrightarrow{+15} \boxed{30}.$$

Another is

$$0 \xrightarrow{+5} 5 \xrightarrow{-10} 5 \xrightarrow{+12} 17 \xrightarrow{+13} \boxed{30},$$

and the third is

$$0 \xrightarrow{-5} 0 \xrightarrow{-10} 0 \xrightarrow{+12} 12 \xrightarrow{-13} 12 \xrightarrow{-15} 12 \xrightarrow{+18} \boxed{30}.$$

Bounding/promising conditions. For current sum s , next item w_{k+1} , and total remaining sum r :

$$\boxed{s + r \geq m}$$

is necessary; otherwise even taking everything left cannot reach the target.

Also, for the next inclusion branch,

$$\boxed{s + w_{k+1} \leq m}$$

must hold; otherwise the next inclusion already overshoots the target.

A standard recursive form is:

```

SubsetSum(s, k, r):
    if s == m:
        print current subset
        return
    if k == n:
        return

```

```

if s + r < m:
    return

if s + w[k] <= m:
    choose w[k]
    SubsetSum(s + w[k], k+1, r - w[k])
    unchoose w[k]

SubsetSum(s, k+1, r - w[k])

```

Branches with $s > 30$ or $s + r < 30$ are pruned.

Q5(c) [6 marks]

For the 0/1-knapsack instance below with capacity $W = 10$ kg, compare backtracking and branch and bound by their state-space trees.

Item	Weight	Profit
A	2	40
B	3.14	50
C	1.98	100
D	5	95
E	3	30

Answer

The optimal subset is

$$\{A, C, D\}$$

with

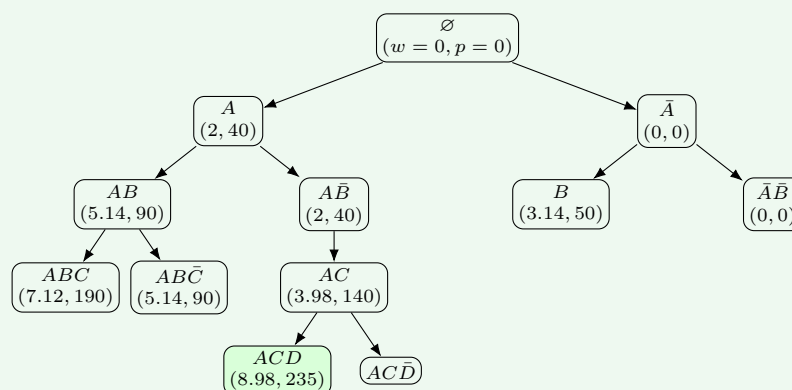
$$\text{weight} = 2 + 1.98 + 5 = 8.98 \leq 10,$$

$$\text{profit} = 40 + 100 + 95 = \boxed{235}.$$

Important Clarification

The exam scan appears to say “prove that backtracking does better than branch and bound.” For this standard knapsack example, the intended textbook point is the opposite: branch and bound can prune more than simple feasibility-only backtracking because it uses an optimistic profit bound. The same numerical instance is commonly used to illustrate this improvement. The solution below follows the standard algorithmic interpretation.

Backtracking tree idea. In the original item order A, B, C, D, E , a feasibility-only DFS can prune only when weight exceeds 10. Many feasible but low-profit nodes must still be explored because feasibility alone does not prove that they cannot beat the optimum.

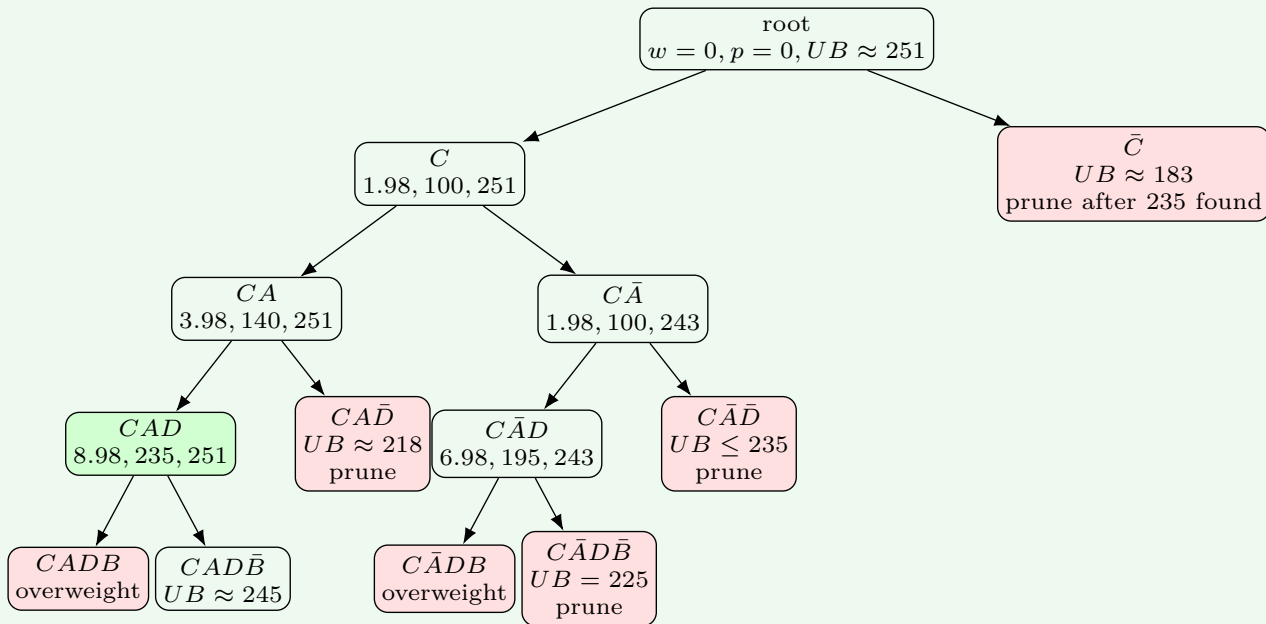


The omitted descendants are still numerous; a branch is killed immediately only when its accumulated weight exceeds 10.

Branch-and-bound tree. First sort by profit/weight ratio:

$$C \ (50.51), \ A \ (20), \ D \ (19), \ B \ (15.92), \ E \ (10).$$

At each node use the fractional-knapsack value as an upper bound UB . A compact best-first tree is:



Once profit 235 is found, every live node with $UB \leq 235$ is discarded. This extra bound-based pruning is exactly what branch and bound adds beyond plain backtracking.

6 Question 6 - NP Problems, Reductions, and Circle Geometry

Q6(a) [4 marks]

Discuss the steps that are used to solve/prove a problem as an NP problem.

Answer

The standard exam interpretation is the workflow for establishing NP-completeness.

1. **Write the decision version.** State clearly what constitutes a YES instance.
2. **Show membership in NP.** Give a certificate and a polynomial-time verifier.
3. **Choose a known NP-complete problem A .**
4. **Construct a polynomial reduction $A \leq_p B$,** where B is the new problem.
5. **Prove correctness of the reduction:**

$$x \in A \iff f(x) \in B.$$

6. Conclude that B is NP-hard; together with $B \in NP$, conclude

$$B \text{ is NP-complete.}$$

Q6(b) [3 marks]

Discuss the idea of a polynomial reduction algorithm to convert problem A to another problem B .

Answer

A polynomial-time reduction from A to B , written

$$A \leq_p B,$$

is an algorithm f computable in polynomial time such that

$$x \in A \iff f(x) \in B.$$

To solve an instance x of A using a solver for B :

$$x \xrightarrow{\text{poly-time } f} f(x) \xrightarrow{\text{solve } B} \text{answer for } A.$$

Therefore, if

$$A \leq_p B \quad \text{and} \quad B \in P,$$

then

$$A \in P.$$

For an NP-hardness proof, the direction is crucial: reduce a *known hard problem* A to the target B , not the other way around.

Q6(c) [7 marks]

Two circles are given as

$$A = (A_x, A_y, A_r), \quad B = (B_x, B_y, B_r),$$

where the first two values are center coordinates and the last value is the radius. Write:

- (i) an algorithm to determine whether circles A and B intersect;
- (ii) an algorithm to determine whether one circle lies inside the other.

Answer

Let

$$d^2 = (A_x - B_x)^2 + (A_y - B_y)^2.$$

Using squared distances avoids computing a square root.

(i) Circumference intersection. Two circles meet (including tangency) exactly when

$$|A_r - B_r| \leq d \leq A_r + B_r.$$

Equivalently,

$$(A_r - B_r)^2 \leq d^2 \leq (A_r + B_r)^2.$$

```
CircleIntersect(A, B):
    d2 = (Ax-Bx)^2 + (Ay-By)^2
    return (Ar-Br)^2 <= d2 and d2 <= (Ar+Br)^2
```

(ii) One circle inside the other. Circle A is inside B exactly when

$$d + A_r \leq B_r.$$

For a square-root-free test, first require $B_r \geq A_r$, then test

$$d^2 \leq (B_r - A_r)^2.$$

Similarly, B is inside A if $A_r \geq B_r$ and

$$d^2 \leq (A_r - B_r)^2.$$

```
Inside(A, B):
    d2 = (Ax-Bx)^2 + (Ay-By)^2
    if Br >= Ar and d2 <= (Br-Ar)^2:
        return "A inside B"
    if Ar >= Br and d2 <= (Ar-Br)^2:
        return "B inside A"
    return "neither contains the other"
```

Each algorithm uses only a constant number of arithmetic operations, so the running time is $O(1)$.

7 Question 7 - Online Algorithms and List Update

Q7(a) [3 marks]

Mention the features of an online algorithm. Define the competitive ratio.

Answer

An **online algorithm** receives requests one at a time and must make each decision without knowing future requests.

Important features:

- input arrives sequentially;
- decisions are made immediately/irrevocably using only past and present information;
- future input is unavailable;
- performance is compared with an optimal offline algorithm OPT that knows the full request sequence;
- worst-case analysis is commonly used.

For a minimisation problem, algorithm A is c -competitive if there exists a constant α such that for every request sequence σ ,

$$A(\sigma) \leq cOPT(\sigma) + \alpha.$$

Ignoring the additive constant for a long sequence, the competitive ratio is the worst-case ratio

$$\sup_{\sigma} \frac{A(\sigma)}{OPT(\sigma)}.$$

Smaller is better; 1 would match the offline optimum.

Q7(b) [11 marks]

In the online list-update problem, the access cost of an item is proportional to its distance from the head of a linked list. Initially

$$\Gamma = (1, 2, 3, \dots, n),$$

and requests arrive as $i_1, i_2, i_3, \dots$. After an access, consider a strategy that moves the accessed item forward in the list.

- Calculate/analyse the competitive ratio for the strategy that moves the accessed item forward by two positions.
- Prove that no deterministic online list-update algorithm has a competitive ratio better than 2 (asymptotically).

Answer

(i) Move-Ahead-Two is not constant-competitive.

Let the last three items initially be

$$a = n - 2, \quad b = n - 1, \quad c = n.$$

Use the repeating adversarial sequence

$$\sigma = (c, b, a, c, b, a, \dots).$$

Initially the tail block is

$$[a, b, c].$$

Request c : it is at position n , costs n , and moving it forward two positions changes the tail block to

$$[c, a, b].$$

Next request b : again it is at position n , costs n , and the tail becomes

$$[b, c, a].$$

Next request a : again cost n , and the tail returns to

$$[a, b, c].$$

Thus this repeats forever, so Move-Ahead-Two pays approximately

$$A(\sigma) = n \cdot |\sigma|.$$

An offline optimum, knowing that only a, b, c are repeatedly requested, can keep them in the first three positions; after an initial rearrangement its access cost is at most 3 per request. Hence for a long sequence

$$\frac{A(\sigma)}{OPT(\sigma)} \gtrsim \frac{n}{3}.$$

Therefore

Move-Ahead-Two has $\Omega(n)$ competitive ratio and is not constant-competitive.

(ii) Deterministic lower bound approaching 2.

Consider any deterministic online algorithm A . An adversary repeatedly requests the item currently at the *last* position of A 's list. Therefore A pays

$$n$$

for every request. For a sequence of m requests,

$$A(\sigma) = mn.$$

Now compare with a fixed offline ordering. If we choose a uniformly random permutation of the n items, every requested item has expected position

$$\frac{n+1}{2}.$$

Therefore, by averaging, there exists at least one fixed ordering whose total cost on the same request sequence is at most

$$\frac{m(n+1)}{2}.$$

The true offline optimum can only be better, apart from an additive initial rearrangement cost. Hence, for long sequences,

$$\frac{A(\sigma)}{OPT(\sigma)} \geq \frac{mn}{m(n+1)/2} = \frac{2n}{n+1} = 2 - \frac{2}{n+1}.$$

As $n \rightarrow \infty$,

$$\frac{2n}{n+1} \rightarrow 2.$$

Thus no deterministic online algorithm can guarantee a competitive ratio strictly below 2 for all list sizes:

deterministic lower bound = 2 asymptotically.

Last-Minute Revision Sheet

Topic	Must remember
Linear probing	$h(k, i) = (h'(k) + i) \bmod m$; primary clustering
Quadratic probing	$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$; reduces primary clustering
Hash collision Q1(b)	$682 \bmod 9 = 7$; index 7 already occupied
Rolling hash Q1(c)	Distinct triples collide at hashes 4, 5, 6
Nonoverlappable pattern	No nonempty proper prefix is also a suffix
Longest prefix of X / suffix of Y	Compute prefix function of $X\#Y$; answer is final π value
Cross product	Sign gives relative orientation; Q3(b): $-, +, -, +$
Graham Scan	Sort by polar angle; pop while turn is non-left
Extended Euclid	$(899, 493) \rightarrow (d, x, y) = (29, -6, 11)$
Totient	$\phi(7000) = 2400$
RSA	$(e, n) = (7, 187)$, $(d, n) = (23, 187)$, $88 \rightarrow 11 \rightarrow 88$
Subset Sum	Bounds: $s + r \geq m$ and next inclusion must not overshoot
Knapsack	Optimum $\{A, C, D\}$, weight 8.98, profit 235
NP-complete proof	In NP + known NP-C reduction $A \leq_p B$ + iff correctness
Circle intersection	$(r_A - r_B)^2 \leq d^2 \leq (r_A + r_B)^2$
Online algorithm	No future knowledge; compare against offline OPT
Competitive ratio	$A(\sigma) \leq cOPT(\sigma) + \alpha$
Move-Ahead-Two	Adversarial last-three cycle gives $\Omega(n)$ ratio
List-update lower bound	No deterministic algorithm beats 2 asymptotically